

6. Neural Networks I

28.10.2025

ETH zürich

D MATH

In today's class...

6.1 A brief history of neural networks

6.2 Feedforward Neural Networks (FNNs)

- Definition
- Training
- Initialization
- Batch-normalization
- Regularization

6.3 The Universal Approximation Property (UAP)

6.4 Complexity bounds for shallow networks

6.5 Complexity bounds for deep networks

6.1 A brief history of neural networks

- 1940s-1960s: Cybernetics
 - first mathematical models of biological neurons [McCulloch and Pitts, 1943, Hebb, 2005]
 - perceptron model [Rosenblatt, 1958]
- 1980s-1990s: Connectionism
 - back-propagation algorithm [Rumelhart et al., 1986]
 - long short-term memory (LSTM) architecture [Hochreiter and Schmidhuber, 1997]
- 2006-now: Deep learning revolution
 - efficient training of deep belief networks [Hinton et al., 2006]
 - efficient training of deep neural networks [Bengio et al., 2006, Ranzato et al., 2006]

6.1 A brief history of neural networks

- 1940s-1960s: Cybernetics
 - first mathematical models of biological neurons [McCulloch and Pitts, 1943, Hebb, 2005]
 - perceptron model [Rosenblatt, 1958]
- 1980s-1990s: Connectionism
 - back-propagation algorithm [Rumelhart et al., 1986]
 - long short-term memory (LSTM) architecture [Hochreiter and Schmidhuber, 1997]
- 2006-now: Deep learning revolution
 - efficient training of deep belief networks [Hinton et al., 2006]
 - efficient training of deep neural networks [Bengio et al., 2006, Ranzato et al., 2006]

A few remarks:

- Neural networks have been known since the 1980s!

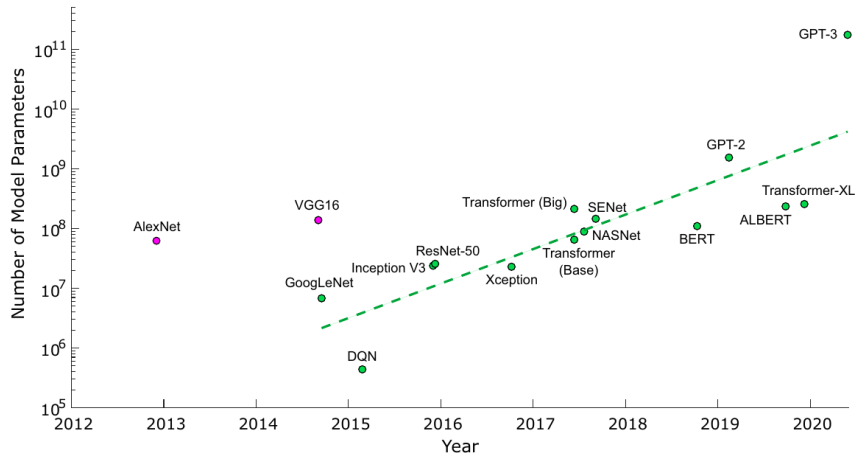
6.1 A brief history of neural networks

- 1940s-1960s: Cybernetics
 - first mathematical models of biological neurons [McCulloch and Pitts, 1943, Hebb, 2005]
 - perceptron model [Rosenblatt, 1958]
- 1980s-1990s: Connectionism
 - back-propagation algorithm [Rumelhart et al., 1986]
 - long short-term memory (LSTM) architecture [Hochreiter and Schmidhuber, 1997]
- 2006-now: Deep learning revolution
 - efficient training of deep belief networks [Hinton et al., 2006]
 - efficient training of deep neural networks [Bengio et al., 2006, Ranzato et al., 2006]

A few remarks:

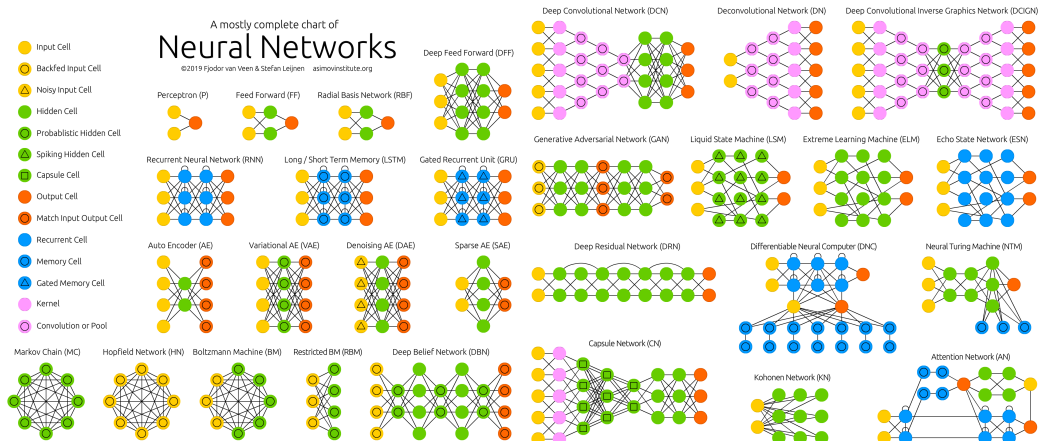
- Neural networks have been known since the 1980s!
- Today's deep learning revolution is mainly due to:
 - increased *computational power* $\implies$ we can train *larger* networks,
 - increased *data availability* $\implies$ the networks can *learn well*.

6.1 A brief history of neural networks



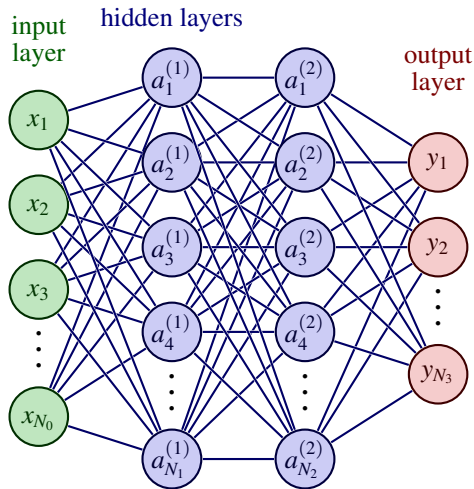
- Neural networks are becoming larger (and especially *deeper*).

6.1 A brief history of neural networks



- Neural networks are becoming *more specialized*.

6.2 Feedforward Neural Networks: definition



- Input layer

$x_j = j$ -th input variable

- First hidden layer

$$\begin{cases} \tilde{a}_i^{(1)} &= \sum_{j=1}^{N_0} W_{i,j}^{(1)} x_j + b_i^{(1)} \\ a_i^{(1)} &= \rho(\tilde{a}_i^{(1)}) \end{cases}$$

- Second hidden layer

$$\begin{cases} \tilde{a}_i^{(2)} &= \sum_{j=1}^{N_1} W_{i,j}^{(2)} a_j^{(1)} + b_i^{(2)} \\ a_i^{(2)} &= \rho(\tilde{a}_i^{(2)}) \end{cases}$$

- Output layer $y_i = \sum_{j=1}^{N_2} W_{i,j}^{(3)} a_j^{(2)} + b_i^{(3)}$

We call $a^{(k)}$ the **activations** and $\tilde{a}^{(k)}$ the **pre-activations**.

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network (FNN)** is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network** (FNN) is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

where:

- each $F^{(k)} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ is an affine function of the form

$$F^{(k)}(x) = W^{(k)} \cdot x + b^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$ are **weights** and $b^{(k)} \in \mathbb{R}^{N_k}$ are **biases**,

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network (FNN)** is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

where:

- each $F^{(k)} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ is an affine function of the form

$$F^{(k)}(x) = W^{(k)} \cdot x + b^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$ are **weights** and $b^{(k)} \in \mathbb{R}^{N_k}$ are **biases**,

- N_k is the **number of neurons** in the k -th layer and $(N_0, \dots, N_L) \in \mathbb{N}^{L+1}$ is the network's **architecture**,

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network (FNN)** is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

where:

- each $F^{(k)} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ is an affine function of the form

$$F^{(k)}(x) = W^{(k)} \cdot x + b^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$ are **weights** and $b^{(k)} \in \mathbb{R}^{N_k}$ are **biases**,

- N_k is the **number of neurons** in the k -th layer and $(N_0, \dots, N_L) \in \mathbb{N}^{L+1}$ is the network's **architecture**,
- $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$ are the network **parameters**,

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network (FNN)** is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

where:

- each $F^{(k)} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ is an affine function of the form

$$F^{(k)}(x) = W^{(k)} \cdot x + b^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$ are **weights** and $b^{(k)} \in \mathbb{R}^{N_k}$ are **biases**,

- N_k is the **number of neurons** in the k -th layer and $(N_0, \dots, N_L) \in \mathbb{N}^{L+1}$ is the network's **architecture**,
- $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$ are the network **parameters**,
- the **parameter space** is $\mathbb{R}^{P(N_0, \dots, N_L)} \ni \theta$, with $P := P(N_0, \dots, N_L) = \sum_{k=1}^L N_k N_{k-1} + N_k$,

6.2 Feedforward Neural Networks: definition

Definition (Feedforward Neural Network)

A **Feedforward Neural Network (FNN)** is a function $F_\theta : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^{N_L}$ defined as

$$F_\theta = \underbrace{F^{(L)}}_{L\text{-th layer}} \circ \underbrace{\rho \odot F^{(L-1)}}_{(L-1)\text{-th layer}} \circ \dots \circ \underbrace{\rho \odot F^{(1)}}_{1\text{-st layer}}$$

where:

- each $F^{(k)} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$ is an affine function of the form

$$F^{(k)}(x) = W^{(k)} \cdot x + b^{(k)},$$

where $W^{(k)} \in \mathbb{R}^{N_k \times N_{k-1}}$ are **weights** and $b^{(k)} \in \mathbb{R}^{N_k}$ are **biases**,

- N_k is the **number of neurons** in the k -th layer and $(N_0, \dots, N_L) \in \mathbb{N}^{L+1}$ is the network's **architecture**,
- $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$ are the network **parameters**,
- the **parameter space** is $\mathbb{R}^{P(N_0, \dots, N_L)} \ni \theta$, with $P := P(N_0, \dots, N_L) = \sum_{k=1}^L N_k N_{k-1} + N_L$,
- $\rho : \mathbb{R} \rightarrow \mathbb{R}$ is a (typically) non-linear function (the **activation function**) applied to vectors element-wise (see symbol $\odot$, above).

6.2 Feedforward Neural Networks: definition

Remarks:

- If we want the network output to be in $\mathbb{R}_+$ or $(0, 1)$, we can add a final transformation φ to the last layer, i.e. $F^{(L)}(x) = \varphi(W^{(L)}x + b^{(L)})$
- Popular choices for φ are the logistic function or shifted tanh (for output in $(0, 1)$) and ReLU/softplus/abs (for output in $\mathbb{R}_+$)
- In the rest of this class we assume that the last layer is affine

6.2 Feedforward Neural Networks: definition

Remarks:

- If we want the network output to be in $\mathbb{R}_+$ or $(0, 1)$, we can add a final transformation φ to the last layer, i.e. $F^{(L)}(x) = \varphi(W^{(L)}x + b^{(L)})$
- Popular choices for φ are the logistic function or shifted tanh (for output in $(0, 1)$) and ReLU/softplus/abs (for output in $\mathbb{R}_+$)
- In the rest of this class we assume that the last layer is affine

Some definitions:

- $k = 0$ is the *input layer*, $k = L$ is the *output layer*, $k \in \{1, \dots, L - 1\}$ are the *hidden layers*,

6.2 Feedforward Neural Networks: definition

Remarks:

- If we want the network output to be in $\mathbb{R}_+$ or $(0, 1)$, we can add a final transformation φ to the last layer, i.e. $F^{(L)}(x) = \varphi(W^{(L)}x + b^{(L)})$
- Popular choices for φ are the logistic function or shifted tanh (for output in $(0, 1)$) and ReLU/softplus/abs (for output in $\mathbb{R}_+$)
- In the rest of this class we assume that the last layer is affine

Some definitions:

- $k = 0$ is the *input layer*, $k = L$ is the *output layer*, $k \in \{1, \dots, L - 1\}$ are the *hidden layers*,
- $L + 1$ is the *number of layers* and L is the *depth*,

6.2 Feedforward Neural Networks: definition

Remarks:

- If we want the network output to be in $\mathbb{R}_+$ or $(0, 1)$, we can add a final transformation φ to the last layer, i.e. $F^{(L)}(x) = \varphi(W^{(L)}x + b^{(L)})$
- Popular choices for φ are the logistic function or shifted tanh (for output in $(0, 1)$) and ReLU/softplus/abs (for output in $\mathbb{R}_+$)
- In the rest of this class we assume that the last layer is affine

Some definitions:

- $k = 0$ is the **input layer**, $k = L$ is the **output layer**, $k \in \{1, \dots, L - 1\}$ are the **hidden layers**,
- $L + 1$ is the **number of layers** and L is the **depth**,
- the network is **shallow** if $L = 2$ (i.e. only one hidden layer), otherwise it's **deep**.

6.2 Feedforward Neural Networks: definition

Remarks:

- If we want the network output to be in $\mathbb{R}_+$ or $(0, 1)$, we can add a final transformation φ to the last layer, i.e. $F^{(L)}(x) = \varphi(W^{(L)}x + b^{(L)})$
- Popular choices for φ are the logistic function or shifted tanh (for output in $(0, 1)$) and ReLU/softplus/abs (for output in $\mathbb{R}_+$)
- In the rest of this class we assume that the last layer is affine

Some definitions:

- $k = 0$ is the **input layer**, $k = L$ is the **output layer**, $k \in \{1, \dots, L - 1\}$ are the **hidden layers**,
- $L + 1$ is the **number of layers** and L is the **depth**,
- the network is **shallow** if $L = 2$ (i.e. only one hidden layer), otherwise it's **deep**.
- $\|N\|_\infty = \max_{0 \leq k \leq L} N_k$ is the **width** of the network.

6.2 Feedforward Neural Networks: definition

- We denote by $\mathcal{H}_{(N_0, N_1, \dots, N_L)}^\rho$ the family of all FNNs with architecture $(N_0, N_1, \dots, N_L)$ and activation function ρ .

6.2 Feedforward Neural Networks: definition

- We denote by $\mathcal{H}_{(N_0, N_1, \dots, N_L)}^\rho$ the family of all FNNs with architecture $(N_0, N_1, \dots, N_L)$ and activation function ρ .

Example

$\mathcal{H}_{(d, m)}^\rho$ is the class of all affine functions $F : \mathbb{R}^d \rightarrow \mathbb{R}^m$.

6.2 Feedforward Neural Networks: definition

- We denote by $\mathcal{H}_{(N_0, N_1, \dots, N_L)}^\rho$ the family of all FNNs with architecture $(N_0, N_1, \dots, N_L)$ and activation function ρ .

Example

$\mathcal{H}_{(d,m)}^\rho$ is the class of all affine functions $F : \mathbb{R}^d \rightarrow \mathbb{R}^m$.

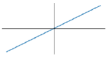
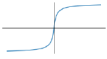
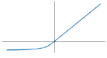
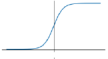
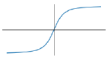
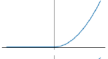
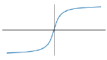
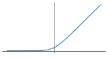
Example

$\mathcal{H}_{(d,n,1)}^\rho$ is the class of all shallow networks with d -dimensional input and one-dimensional output, i.e. FNNs of the form:

$$x \mapsto \sum_{i=1}^n W_{1,i}^{(2)} \rho \left(\sum_{j=1}^d W_{i,j}^{(1)} x_j + b_i^{(1)} \right) + b^{(2)}$$

6.2 Feedforward Neural Networks: definition

Commonly used^{3 4} activation functions:

linear	x		softsign	$\frac{x}{1+ x }$	
Heaviside / step function	$1_{(0,\infty)}(x)$		inverse square root linear unit	$x \cdot 1_{[0,\infty)}(x) +$ $\frac{x}{\sqrt{1+ax^2}} \cdot 1_{(-\infty,0)}(x)$ for $a > 0$	
logistic / sigmoid	$\frac{1}{1+e^{-x}}$		inverse square root unit	$\frac{x}{\sqrt{1+ax^2}}$ for $a > 0$	
rectified linear unit (ReLU)	$\max\{0, x\}$		tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	
power rectified linear unit	$\max\{0, x\}^k$ for $k \in \mathbb{N}$		arctan	$\arctan(x)$	
parametric ReLU (PReLU)	$\max\{ax, x\}$ for $a \geq 0, a \neq 1$		softplus	$\ln(1 + e^x)$	
exponential linear unit (ELU)	$x \cdot 1_{[0,\infty)}(x) +$ $(e^x - 1) \cdot 1_{(-\infty,0)}(x)$				

³Table taken from [Berner et al., 2021]

⁴Many other choices are possible. $\rho(x) = \log(e^{ax} + e^x)$ also works well empirically [Cheridito and Gersey, 2023]

6.2 Feedforward Neural Networks: training

6.2 Feedforward Neural Networks: training

Reminder

Recall the Empirical Risk Minimization (ERM) problem.

Given

- *a training set $D = ((x_i, y_i), i = 1, \dots, m)$ of m iid realizations of (X, Y) ,*
- *a loss function $\ell(\hat{y}, y)$*
- *an hypothesis class $\mathcal{H}$,*

we want to solve:

$$\inf_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i).$$

6.2 Feedforward Neural Networks: training

Reminder

Recall the Empirical Risk Minimization (ERM) problem.

Given

- *a training set $D = ((x_i, y_i), i = 1, \dots, m)$ of m iid realizations of (X, Y) ,*
- *a loss function $\ell(\hat{y}, y)$*
- *an hypothesis class $\mathcal{H}$,*

we want to solve:

$$\inf_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell(f(x_i), y_i).$$

For $\mathcal{H} = \mathcal{H}_{(N_0, \dots, N_L)}^p$ we have:

$$\inf_{F_\theta \in \mathcal{H}_{(N_0, \dots, N_L)}^p} \frac{1}{m} \sum_{i=1}^m \ell(F_\theta(x_i), y_i) = \inf_{\theta \in \mathbb{R}^{p(N_0, \dots, N_L)}} \underbrace{\frac{1}{m} \sum_{i=1}^m \ell(F_\theta(x_i), y_i)}_{\mathcal{L}(\theta, D)} \quad (\star)$$

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

- the parameter space $\mathbb{R}^{P(N_0, \dots, N_L)}$ can be *very* high dimensional,

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

- the parameter space $\mathbb{R}^{P(N_0, \dots, N_L)}$ can be *very* high dimensional,
- if $\theta \mapsto \mathcal{L}(\theta, D)$ is a.e. differentiable, then we can use (Stochastic) Gradient Descent:

$$\theta_0 \sim \mathbb{P}_{\text{initialization}}, \quad \theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta, D)|_{\theta=\theta_t}$$

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

- the parameter space $\mathbb{R}^{P(N_0, \dots, N_L)}$ can be **very** high dimensional,
- if $\theta \mapsto \mathcal{L}(\theta, D)$ is a.e. differentiable, then we can use (Stochastic) Gradient Descent:

$$\theta_0 \sim \mathbb{P}_{\text{initialization}}, \quad \theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta, D)|_{\theta=\theta_t}$$

- $\theta \mapsto \mathcal{L}(\theta, D)$ is typically non-convex $\implies$ convergence to a global minimum is **not** guaranteed,

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

- the parameter space $\mathbb{R}^{P(N_0, \dots, N_L)}$ can be **very** high dimensional,
- if $\theta \mapsto \mathcal{L}(\theta, D)$ is a.e. differentiable, then we can use (Stochastic) Gradient Descent:

$$\theta_0 \sim \mathbb{P}_{\text{initialization}}, \quad \theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta, D)|_{\theta=\theta_t}$$

- $\theta \mapsto \mathcal{L}(\theta, D)$ is typically non-convex $\implies$ convergence to a global minimum is **not** guaranteed,
- computing $\nabla_{\theta} \mathcal{L}(\theta, D)$ can be done efficiently even in large networks using **back-propagation**,

6.2 Feedforward Neural Networks: training

The optimization problem $(\star)$ is highly non-trivial:

- the parameter space $\mathbb{R}^{P(N_0, \dots, N_L)}$ can be **very** high dimensional,
- if $\theta \mapsto \mathcal{L}(\theta, D)$ is a.e. differentiable, then we can use (Stochastic) Gradient Descent:

$$\theta_0 \sim \mathbb{P}_{\text{initialization}}, \quad \theta_{t+1} = \theta_t - \eta_t \nabla_{\theta} \mathcal{L}(\theta, D)|_{\theta=\theta_t}$$

- $\theta \mapsto \mathcal{L}(\theta, D)$ is typically non-convex $\implies$ convergence to a global minimum is **not** guaranteed,
- computing $\nabla_{\theta} \mathcal{L}(\theta, D)$ can be done efficiently even in large networks using **back-propagation**,
- during training, each network layer learns to produce features that are useful for the next layer to minimize the loss.

6.2 Feedforward Neural Networks: training

Algorithm: Forward propagation

Input: Parameters $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$; datapoint (x, y)

Output: Loss value $\ell(F_\theta(x), y)$

$a^{(0)} := x;$

for $k = 1, \dots, L - 1$ **do**

$\tilde{a}^{(k)} := W^{(k)} a^{(k-1)} + b^{(k)};$

$a^{(k)} := \rho(\tilde{a}^{(k)});$

end

$\hat{y} := W^{(L)} a^{(L-1)} + b^{(L)};$

return $\ell(\hat{y}, y)$

6.2 Feedforward Neural Networks: training

Algorithm: Back-propagation

Input: Parameters $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$; datapoint (x, y)

Output: Gradient $\nabla_{\theta} \ell(F_{\theta}(x), y)$

Compute $\ell(F_{\theta}(x), y)$ using forward propagation;

$\text{grad} \leftarrow \nabla_{\hat{y}} \ell(\hat{y}, y)$;

$\nabla_{W^{(L)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(L-1)T}$;

$\nabla_{b^{(L)}} \ell(\hat{y}, y) = \text{grad}$;

for $k = L - 1, \dots, 1$ **do**

$\text{grad} \leftarrow \nabla_{\tilde{a}^{(k)}} \ell(\hat{y}, y) = (\text{grad}^T \cdot W^{(k+1)} \cdot \text{diag}(\rho'(\tilde{a}^{(k)})))^T$;

$\nabla_{W^{(k)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(k-1)T}$;

$\nabla_{b^{(k)}} \ell(\hat{y}, y) = \text{grad}$;

end

return $((\nabla_{W^{(k)}} \ell(\hat{y}, y), \nabla_{b^{(k)}} \ell(\hat{y}, y)) \text{ for } k = 1, \dots, L)$.

6.2 Feedforward Neural Networks: training

Algorithm: Back-propagation

Input: Parameters $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$; datapoint (x, y)

Output: Gradient $\nabla_{\theta} \ell(F_{\theta}(x), y)$

Compute $\ell(F_{\theta}(x), y)$ using forward propagation;

$\text{grad} \leftarrow \nabla_{\hat{y}} \ell(\hat{y}, y)$;

$\nabla_{W^{(L)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(L-1)T}$;

$\nabla_{b^{(L)}} \ell(\hat{y}, y) = \text{grad}$;

for $k = L - 1, \dots, 1$ **do**

$\text{grad} \leftarrow \nabla_{\tilde{a}^{(k)}} \ell(\hat{y}, y) = (\text{grad}^T \cdot W^{(k+1)} \cdot \text{diag}(\rho'(\tilde{a}^{(k)})))^T$;

$\nabla_{W^{(k)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(k-1)T}$;

$\nabla_{b^{(k)}} \ell(\hat{y}, y) = \text{grad}$;

end

return $((\nabla_{W^{(k)}} \ell(\hat{y}, y), \nabla_{b^{(k)}} \ell(\hat{y}, y)) \text{ for } k = 1, \dots, L)$.

- Compute gradients in the right order (backwards) and store intermediate results (grad)
 $\implies$ complexity is $O(L)$ in time and $O(L)$ in memory.

6.2 Feedforward Neural Networks: training

Algorithm: Back-propagation

Input: Parameters $\theta = ((W^{(k)}, b^{(k)}), k = 1, \dots, L)$; datapoint (x, y)

Output: Gradient $\nabla_{\theta} \ell(F_{\theta}(x), y)$

Compute $\ell(F_{\theta}(x), y)$ using forward propagation;

$\text{grad} \leftarrow \nabla_{\hat{y}} \ell(\hat{y}, y)$;

$\nabla_{W^{(L)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(L-1)T}$;

$\nabla_{b^{(L)}} \ell(\hat{y}, y) = \text{grad}$;

for $k = L - 1, \dots, 1$ **do**

$\text{grad} \leftarrow \nabla_{\tilde{a}^{(k)}} \ell(\hat{y}, y) = (\text{grad}^T \cdot W^{(k+1)} \cdot \text{diag}(\rho'(\tilde{a}^{(k)})))^T$;

$\nabla_{W^{(k)}} \ell(\hat{y}, y) = \text{grad} \cdot a^{(k-1)T}$;

$\nabla_{b^{(k)}} \ell(\hat{y}, y) = \text{grad}$;

end

return $((\nabla_{W^{(k)}} \ell(\hat{y}, y), \nabla_{b^{(k)}} \ell(\hat{y}, y)) \text{ for } k = 1, \dots, L)$.

- Compute gradients in the right order (backwards) and store intermediate results (grad)
 $\implies$ complexity is $O(L)$ in time and $O(L)$ in memory.
- The network stops learning if $\nabla_{\theta} \mathcal{L} \approx 0$
 $\implies$ choose ρ and $\mathcal{L}$ such that $\rho', \nabla_{\hat{y}} \mathcal{L}(\hat{y}, y) \neq 0$ almost everywhere

6.2 Feedforward Neural Networks: training

In practice:

- the gradient is averaged over a batch of n_{batch} datapoints $((x_i, y_i), 1 \leq i \leq n_{\text{batch}})$, where n_{batch} (the *batch size*) is a hyperparameter,

6.2 Feedforward Neural Networks: training

In practice:

- the gradient is averaged over a batch of n_{batch} datapoints $((x_i, y_i), 1 \leq i \leq n_{\text{batch}})$, where n_{batch} (the **batch size**) is a hyperparameter,
- back-propagation (or variations of it) is used for many architectures (e.g. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), etc),

6.2 Feedforward Neural Networks: training

In practice:

- the gradient is averaged over a batch of n_{batch} datapoints $((x_i, y_i), 1 \leq i \leq n_{\text{batch}})$, where n_{batch} (the **batch size**) is a hyperparameter,
- back-propagation (or variations of it) is used for many architectures (e.g. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), etc),
- in ML packages, such as TensorFlow and PyTorch, gradients can be computed automatically using back-propagation on computational graphs

6.2 Feedforward Neural Networks: training

In practice:

- the gradient is averaged over a batch of n_{batch} datapoints $((x_i, y_i), 1 \leq i \leq n_{\text{batch}})$, where n_{batch} (the *batch size*) is a hyperparameter,
- back-propagation (or variations of it) is used for many architectures (e.g. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), etc),
- in ML packages, such as TensorFlow and PyTorch, gradients can be computed automatically using back-propagation on computational graphs
- you can also build custom layers using built-in functions,

6.2 Feedforward Neural Networks: training

In practice:

- the gradient is averaged over a batch of n_{batch} datapoints $((x_i, y_i), 1 \leq i \leq n_{\text{batch}})$, where n_{batch} (the **batch size**) is a hyperparameter,
- back-propagation (or variations of it) is used for many architectures (e.g. Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), etc),
- in ML packages, such as TensorFlow and PyTorch, gradients can be computed automatically using back-propagation on computational graphs
- you can also build custom layers using built-in functions,
- learn more about it here:
 - <https://pytorch.org/docs/stable/autograd.html>
 - https://pytorch.org/tutorials/beginner/basics/autograd_tutorial.html
 - <https://www.tensorflow.org/guide/autodiff>
 - https://www.tensorflow.org/guide/intro_to_graphs

6.2 Feedforward Neural Networks: initialization

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,
- we can use ***random initialization***, but:
 - if θ_0 is too small
 $\implies$ gradients vanish (multiplication of weights on average less than one in chain rule),

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,
- we can use ***random initialization***, but:
 - if θ_0 is too small
 $\implies$ gradients vanish (multiplication of weights on average less than one in chain rule),
 - if θ_0 is too big
 $\implies$ gradients explode (multiplication of weights on average greater than one in chain rule),

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,
- we can use **random initialization**, but:
 - if θ_0 is too small
 $\implies$ gradients vanish (multiplication of weights on average less than one in chain rule),
 - if θ_0 is too big
 $\implies$ gradients explode (multiplication of weights on average greater than one in chain rule),
- one initializer that works well is the **Xavier initialization**:

$$w_{j,l}^{(k)} \sim \mathcal{N}\left(0, \frac{1}{N_k}\right) \text{ iid} \quad \text{and} \quad b^{(k)} = 0, \quad \text{for all } 1 \leq k \leq L,$$

where N_k is the width of the k -th layer.

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,
- we can use **random initialization**, but:
 - if θ_0 is too small
 $\implies$ gradients vanish (multiplication of weights on average less than one in chain rule),
 - if θ_0 is too big
 $\implies$ gradients explode (multiplication of weights on average greater than one in chain rule),
- one initializer that works well is the **Xavier initialization**:

$$W_{j,l}^{(k)} \sim \mathcal{N}\left(0, \frac{1}{N_k}\right) \text{ iid} \quad \text{and} \quad b^{(k)} = 0, \quad \text{for all } 1 \leq k \leq L,$$

where N_k is the width of the k -th layer.

- also other initializers can be used (see `torch.nn.init` and `tf.keras.initializers`),

6.2 Feedforward Neural Networks: initialization

- The critical point reached by SGD in general depends on the initialization θ_0 , because the loss function is not convex in θ ,
- if we initialize all parameters to a common constant c , i.e. $\theta_0 = c$, then all weights have the same gradient and will evolve identically (different neurons will not learn different things)
 $\implies$ we must initialize each weight with a different value,
- we can use **random initialization**, but:
 - if θ_0 is too small
 $\implies$ gradients vanish (multiplication of weights on average less than one in chain rule),
 - if θ_0 is too big
 $\implies$ gradients explode (multiplication of weights on average greater than one in chain rule),
- one initializer that works well is the **Xavier initialization**:

$$W_{j,l}^{(k)} \sim \mathcal{N}\left(0, \frac{1}{N_k}\right) \text{ iid} \quad \text{and} \quad b^{(k)} = 0, \quad \text{for all } 1 \leq k \leq L,$$

where N_k is the width of the k -th layer.

- also other initializers can be used (see `torch.nn.init` and `tf.keras.initializers`),
- typically we repeat the training for different sampled values of θ_0 .

6.2 Feedforward Neural Networks: batch normalization

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).
- **Main idea:** stabilize training by normalizing the activations of each hidden layer using the empirical mean and standard deviation in each batch,

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).
- **Main idea:** stabilize training by normalizing the activations of each hidden layer using the empirical mean and standard deviation in each batch,
- if $(x_i, 1 \leq i \leq n_{\text{batch}})$ is a batch of size n_{batch} and $a_j^{(k)}$ is the activation of the j -th neuron in the k -th layer, then:

$$a_j^{(k)}(x_i) \leftarrow \frac{a_j^{(k)}(x_i) - \mu_j^{(k)}}{\sqrt{\sigma_j^{2(k)} + \varepsilon}},$$

where

$$\mu_j^{(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} a_j^{(k)}(x_i) \quad \text{and} \quad \sigma_j^{2(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (a_j^{(k)}(x_i) - \mu_j^{(k)})^2,$$

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).
- **Main idea:** stabilize training by normalizing the activations of each hidden layer using the empirical mean and standard deviation in each batch,
- if $(x_i, 1 \leq i \leq n_{\text{batch}})$ is a batch of size n_{batch} and $a_j^{(k)}$ is the activation of the j -th neuron in the k -th layer, then:

$$a_j^{(k)}(x_i) \leftarrow \frac{a_j^{(k)}(x_i) - \mu_j^{(k)}}{\sqrt{\sigma_j^{2(k)} + \varepsilon}},$$

where

$$\mu_j^{(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} a_j^{(k)}(x_i) \quad \text{and} \quad \sigma_j^{2(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (a_j^{(k)}(x_i) - \mu_j^{(k)})^2,$$

- $\varepsilon > 0$ is a small positive constant ($\varepsilon \approx 10^{-5}$) to prevent division by zero,

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).
- **Main idea:** stabilize training by normalizing the activations of each hidden layer using the empirical mean and standard deviation in each batch,
- if $(x_i, 1 \leq i \leq n_{\text{batch}})$ is a batch of size n_{batch} and $a_j^{(k)}$ is the activation of the j -th neuron in the k -th layer, then:

$$a_j^{(k)}(x_i) \leftarrow \frac{a_j^{(k)}(x_i) - \mu_j^{(k)}}{\sqrt{\sigma_j^{2(k)} + \varepsilon}},$$

where

$$\mu_j^{(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} a_j^{(k)}(x_i) \quad \text{and} \quad \sigma_j^{2(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (a_j^{(k)}(x_i) - \mu_j^{(k)})^2,$$

- $\varepsilon > 0$ is a small positive constant ($\varepsilon \approx 10^{-5}$) to prevent division by zero,
- we back-propagate the gradient through the batch normalization operations,

6.2 Feedforward Neural Networks: batch normalization

- **Problem:** during training, the parameter updates in one hidden layer often result in the shifting or re-scaling of the input of the next hidden layer, thus slowing training down (*internal covariate shift*).
- **Main idea:** stabilize training by normalizing the activations of each hidden layer using the empirical mean and standard deviation in each batch,
- if $(x_i, 1 \leq i \leq n_{\text{batch}})$ is a batch of size n_{batch} and $a_j^{(k)}$ is the activation of the j -th neuron in the k -th layer, then:

$$a_j^{(k)}(x_i) \leftarrow \frac{a_j^{(k)}(x_i) - \mu_j^{(k)}}{\sqrt{\sigma_j^{2(k)} + \varepsilon}},$$

where

$$\mu_j^{(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} a_j^{(k)}(x_i) \quad \text{and} \quad \sigma_j^{2(k)} = \frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (a_j^{(k)}(x_i) - \mu_j^{(k)})^2,$$

- $\varepsilon > 0$ is a small positive constant ($\varepsilon \approx 10^{-5}$) to prevent division by zero,
- we back-propagate the gradient through the batch normalization operations,
- in PyTorch and TensorFlow batch normalization is implemented as a layer (see `torch.nn.BatchNorm2d` and `tf.keras.layers.BatchNormalization`).

6.2 Feedforward Neural Networks: regularization

6.2 Feedforward Neural Networks: regularization

Loss functions are typically regularized, i.e. we minimize a modified objective:

$$\tilde{\mathcal{L}}(\theta, D) = \mathcal{L}(\theta, D) + \lambda \mathcal{R}(\theta),$$

where λ is an hyper-parameter (to choose via cross-validation on a validation set).

6.2 Feedforward Neural Networks: regularization

Loss functions are typically regularized, i.e. we minimize a modified objective:

$$\tilde{\mathcal{L}}(\theta, D) = \mathcal{L}(\theta, D) + \lambda \mathcal{R}(\theta),$$

where λ is an hyper-parameter (to choose via cross-validation on a validation set).

Popular choices for $\mathcal{R}(\cdot)$ are:

- $\mathcal{R}(\theta) = \|\theta\|_2^2$ (L^2 or Tikhonov regularization, a.k.a. Ridge regression)
 $\implies$ shrinks θ in directions that do not minimize the objective strongly enough

6.2 Feedforward Neural Networks: regularization

Loss functions are typically regularized, i.e. we minimize a modified objective:

$$\tilde{\mathcal{L}}(\theta, D) = \mathcal{L}(\theta, D) + \lambda \mathcal{R}(\theta),$$

where λ is an hyper-parameter (to choose via cross-validation on a validation set).

Popular choices for $\mathcal{R}(\cdot)$ are:

- $\mathcal{R}(\theta) = \|\theta\|_2^2$ (L^2 or Tikhonov regularization, a.k.a. Ridge regression)
 $\implies$ shrinks θ in directions that do not minimize the objective strongly enough
- $\mathcal{R}(\theta) = \|\theta\|_1$ (L^1 regularization or Lasso regression)
 $\implies$ leads to sparse θ

6.2 Feedforward Neural Networks: regularization

Loss functions are typically regularized, i.e. we minimize a modified objective:

$$\tilde{\mathcal{L}}(\theta, D) = \mathcal{L}(\theta, D) + \lambda \mathcal{R}(\theta),$$

where λ is an hyper-parameter (to choose via cross-validation on a validation set).

Popular choices for $\mathcal{R}(\cdot)$ are:

- $\mathcal{R}(\theta) = \|\theta\|_2^2$ (L^2 or Tikhonov regularization, a.k.a. Ridge regression)
 $\implies$ shrinks θ in directions that do not minimize the objective strongly enough
- $\mathcal{R}(\theta) = \|\theta\|_1$ (L^1 regularization or Lasso regression)
 $\implies$ leads to sparse θ

In practice:

- Only the layers' weights W are regularized, because regularizing the biases may cause underfitting,
- Different regularization parameters λ can be used in different layers.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.
- Equivalent to limiting the volume of searchable parameter space, as in L^2 -regularization.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.
- Equivalent to limiting the volume of searchable parameter space, as in L^2 -regularization.

- **Dropout:**

- Before each gradient step, set to zero each neuron with probability p , independently of all others; then perform back-propagation as usual.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.
- Equivalent to limiting the volume of searchable parameter space, as in L^2 -regularization.

- **Dropout:**

- Before each gradient step, set to zero each neuron with probability p , independently of all others; then perform back-propagation as usual.
- Encourages sparse and redundant representations.

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.
- Equivalent to limiting the volume of searchable parameter space, as in L^2 -regularization.

- **Dropout:**

- Before each gradient step, set to zero each neuron with probability p , independently of all others; then perform back-propagation as usual.
- Encourages sparse and redundant representations.
- Approximately equivalent to training exponentially many neural networks (as in *bagging*).

6.2 Feedforward Neural Networks: regularization

Training techniques can act as regularizers too!

- **Early stopping:**

- For fixed $p \in \mathbb{N}$ (the *patience* parameter), stop training if after p SGD steps the error on a validation set has not decreased, then return the parameters that achieved the minimum validation error.
- Equivalent to cross-validation on total number of SGD steps.
- Equivalent to limiting the volume of searchable parameter space, as in L^2 -regularization.

- **Dropout:**

- Before each gradient step, set to zero each neuron with probability p , independently of all others; then perform back-propagation as usual.
- Encourages sparse and redundant representations.
- Approximately equivalent to training exponentially many neural networks (as in *bagging*).

- **Bagging, dataset augmentation, weights robustness, parameter sharing...**

Reminder

Recall the **bias-variance** trade-off:

Given

- a training set $D = ((X_i, Y_i), i = 1, \dots, m)$ of independent copies of (X, Y) ,
- a numerical solution $\hat{f}_m$ to the ERM problem: $\inf_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell(f(X_i), Y_i)$,
- $\hat{f}_m^{\text{avg}}(x) := \mathbb{E}[\hat{f}_m(x)]$, where $\mathbb{E}[\cdot]$ is with respect to the distribution of the sample D ,
- $\bar{f}(x)$, the (unknown) theoretical best predictor, i.e. the minimizer of $\mathbb{E}[\ell(f(X), Y)]$,

we have the decomposition:

$$\mathbb{E} \left[\left(\hat{f}_m(X) - Y \right)^2 \right] = \underbrace{\mathbb{E} \left[\left(\bar{f}(X) - Y \right)^2 \right]}_{\text{irreducible error}} + \underbrace{\mathbb{E} \left[\left(\hat{f}_m^{\text{avg}}(X) - \bar{f}(X) \right)^2 \right]}_{\text{bias}^2} + \underbrace{\mathbb{E} \left[\left(\hat{f}_m(X) - \hat{f}_m^{\text{avg}}(X) \right)^2 \right]}_{\text{variance}}$$

Reminder

Recall the **bias-variance** trade-off:

Given

- a training set $D = ((X_i, Y_i), i = 1, \dots, m)$ of independent copies of (X, Y) ,
- a numerical solution $\hat{f}_m$ to the ERM problem: $\inf_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell(f(X_i), Y_i)$,
- $\hat{f}_m^{\text{avg}}(x) := \mathbb{E}[\hat{f}_m(x)]$, where $\mathbb{E}[\cdot]$ is with respect to the distribution of the sample D ,
- $\bar{f}(x)$, the (unknown) theoretical best predictor, i.e. the minimizer of $\mathbb{E}[\ell(f(X), Y)]$,

we have the decomposition:

$$\mathbb{E} \left[\left(\hat{f}_m(X) - Y \right)^2 \right] = \underbrace{\mathbb{E} \left[\left(\bar{f}(X) - Y \right)^2 \right]}_{\text{irreducible error}} + \underbrace{\mathbb{E} \left[\left(\hat{f}_m^{\text{avg}}(X) - \bar{f}(X) \right)^2 \right]}_{\text{bias}^2} + \underbrace{\mathbb{E} \left[\left(\hat{f}_m(X) - \hat{f}_m^{\text{avg}}(X) \right)^2 \right]}_{\text{variance}}$$

- Today we focus on the **bias** term: can neural networks approximate functions arbitrarily well?

6.3 Universal Approximation Property (UAP)

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

- Assume that $\rho \in C^\infty(\mathbb{R})$:

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

- Assume that $\rho \in C^\infty(\mathbb{R})$:
 - We have that $\frac{\rho((w+h)x+b) - \rho(wx+b)}{h} \in \mathcal{H}^\rho$ and therefore $\frac{d}{dw} \rho(wx+b) \in \overline{\mathcal{H}^\rho}$.

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

- Assume that $\rho \in C^\infty(\mathbb{R})$:
 - We have that $\frac{\rho((w+h)x+b) - \rho(wx+b)}{h} \in \mathcal{H}^\rho$ and therefore $\frac{d}{dw} \rho(wx+b) \in \overline{\mathcal{H}^\rho}$.
 - Iterating the same argument, it follows that $\frac{d^k}{dw^k} \rho(wx+b) \in \overline{\mathcal{H}^\rho}$ for every $k \in \mathbb{N}$.

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

- Assume that $\rho \in C^\infty(\mathbb{R})$:
 - We have that $\frac{\rho((w+h)x+b) - \rho(wx+b)}{h} \in \mathcal{H}^\rho$ and therefore $\frac{d}{dw}\rho(wx+b) \in \overline{\mathcal{H}}^\rho$.
 - Iterating the same argument, it follows that $\frac{d^k}{dw^k}\rho(wx+b) \in \overline{\mathcal{H}}^\rho$ for every $k \in \mathbb{N}$.
 - Since ρ is not a polynomial, for every k there exists a $\theta_k \in \mathbb{R}$ such that $\rho^{(k)}(\theta_k) \neq 0$ and
$$x^k \rho^{(k)}(\theta_k) = \left. \frac{d^k}{dw^k} \rho(wx+b) \right|_{w=0, b=\theta_k} \in \overline{\mathcal{H}}^\rho.$$

6.3 Universal Approximation Property (UAP)

- The first UAP proof was due to [Cybenko, 1989],
- [Leshno et al., 1993] gave a sufficient and necessary condition on ρ for the UAP to hold.

Theorem 1 (UAP [Leshno et al., 1993])

Let $K \subseteq \mathbb{R}^d$ be compact and $\rho : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous activation function, then $\bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$ is dense in $C(K)$ with the supremum norm if and only if ρ is not a polynomial.

Proof.

Let $\mathcal{H}^\rho := \bigcup_{n \in \mathbb{N}} \mathcal{H}_{(d,n,1)}^\rho$. Without loss of generality, it suffices to prove it for $d = 1$ (see Problem Set 4, Ex 8).

- Assume that $\rho \in C^\infty(\mathbb{R})$:
 - We have that $\frac{\rho((w+h)x+b) - \rho(wx+b)}{h} \in \mathcal{H}^\rho$ and therefore $\frac{d}{dw}\rho(wx+b) \in \overline{\mathcal{H}}^\rho$.
 - Iterating the same argument, it follows that $\frac{d^k}{dw^k}\rho(wx+b) \in \overline{\mathcal{H}}^\rho$ for every $k \in \mathbb{N}$.
 - Since ρ is not a polynomial, for every k there exists a $\theta_k \in \mathbb{R}$ such that $\rho^{(k)}(\theta_k) \neq 0$ and
$$x^k \rho^{(k)}(\theta_k) = \left. \frac{d^k}{dw^k}\rho(wx+b) \right|_{w=0, b=\theta_k} \in \overline{\mathcal{H}}^\rho.$$
 - This implies that $\overline{\mathcal{H}}^\rho$ contains all monomials and therefore all polynomials. By the Stone-Weierstrass theorem $\overline{\mathcal{H}}^\rho = C(K)$.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument. Furthermore $\rho * \phi \in \overline{\mathcal{H}}^\rho$ (by representing the convolution as the limit of Riemannian sums), therefore $\mathcal{H}^{\rho * \phi} \subseteq \overline{\mathcal{H}}^\rho$, which implies $\overline{\mathcal{H}}^\rho = C(K)$.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument. Furthermore $\rho * \phi \in \overline{\mathcal{H}}^\rho$ (by representing the convolution as the limit of Riemannian sums), therefore $\mathcal{H}^{\rho * \phi} \subseteq \overline{\mathcal{H}}^\rho$, which implies $\overline{\mathcal{H}}^\rho = C(K)$.
 - Finally, one can show that if ρ is not a polynomial, then there exists always a $\phi \in C_c^\infty(\mathbb{R})$ such that $\rho * \phi$ is not a polynomial (see Problem Set 4, Ex 9), therefore the previous step holds for every $\rho \in C(\mathbb{R})$.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument. Furthermore $\rho * \phi \in \overline{\mathcal{H}^\rho}$ (by representing the convolution as the limit of Riemannian sums), therefore $\mathcal{H}^{\rho * \phi} \subseteq \overline{\mathcal{H}^\rho}$, which implies $\overline{\mathcal{H}^\rho} = C(K)$.
 - Finally, one can show that if ρ is not a polynomial, then there exists always a $\phi \in C_c^\infty(\mathbb{R})$ such that $\rho * \phi$ is not a polynomial (see Problem Set 4, Ex 9), therefore the previous step holds for every $\rho \in C(\mathbb{R})$.
- For the reverse implication (ρ is a polynomial $\implies \overline{\mathcal{H}^\rho} \neq C(K)$), notice that if ρ is a polynomial of degree k , then $\overline{\mathcal{H}^\rho}$ is the set of polynomials of degree k , which is not dense in $C(K)$.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument. Furthermore $\rho * \phi \in \overline{\mathcal{H}}^\rho$ (by representing the convolution as the limit of Riemannian sums), therefore $\mathcal{H}^{\rho * \phi} \subseteq \overline{\mathcal{H}}^\rho$, which implies $\overline{\mathcal{H}}^\rho = C(K)$.
 - Finally, one can show that if ρ is not a polynomial, then there exists always a $\phi \in C_c^\infty(\mathbb{R})$ such that $\rho * \phi$ is not a polynomial (see Problem Set 4, Ex 9), therefore the previous step holds for every $\rho \in C(\mathbb{R})$.
- For the reverse implication (ρ is a polynomial $\implies \overline{\mathcal{H}}^\rho \neq C(K)$), notice that if ρ is a polynomial of degree k , then $\overline{\mathcal{H}}^\rho$ is the set of polynomials of degree k , which is not dense in $C(K)$.



Remarks:

- Holds also for ρ continuous a.e. and locally bounded (i.e. $\rho \in L^\infty(K), \forall K \subseteq \mathbb{R}$ compact) $\implies$ applies to all commonly used activation functions.

6.3 Universal Approximation Property (UAP)

Proof.

- For generic $\rho \in C(\mathbb{R})$, we need a mollification argument:
 - Suppose there exists $\phi \in C_c^\infty(\mathbb{R})$ such that $(\rho * \phi)(x) = \int \rho(x - y)\phi(y)dy$ is not a polynomial. Then $\rho * \phi \in C^\infty(\mathbb{R})$ and $\overline{\mathcal{H}^{\rho * \phi}} = C(K)$ by the previous argument. Furthermore $\rho * \phi \in \overline{\mathcal{H}}^\rho$ (by representing the convolution as the limit of Riemannian sums), therefore $\mathcal{H}^{\rho * \phi} \subseteq \overline{\mathcal{H}}^\rho$, which implies $\overline{\mathcal{H}}^\rho = C(K)$.
 - Finally, one can show that if ρ is not a polynomial, then there exists always a $\phi \in C_c^\infty(\mathbb{R})$ such that $\rho * \phi$ is not a polynomial (see Problem Set 4, Ex 9), therefore the previous step holds for every $\rho \in C(\mathbb{R})$.
- For the reverse implication (ρ is a polynomial $\implies \overline{\mathcal{H}}^\rho \neq C(K)$), notice that if ρ is a polynomial of degree k , then $\overline{\mathcal{H}}^\rho$ is the set of polynomials of degree k , which is not dense in $C(K)$.



Remarks:

- Holds also for ρ continuous a.e. and locally bounded (i.e. $\rho \in L^\infty(K), \forall K \subseteq \mathbb{R}$ compact) $\implies$ applies to all commonly used activation functions.
- The theorem gives the important equivalence: $\text{UAP} \iff \text{non-polynomial activation}$.

6.4 Complexity bounds for shallow networks

- **Question:** in practice, for a shallow neural network with architecture $(1, N, 1)$, how many hidden neurons N do I need to approximate f with a given accuracy ε ?

6.4 Complexity bounds for shallow networks

- **Question:** in practice, for a shallow neural network with architecture $(1, N, 1)$, how many hidden neurons N do I need to approximate f with a given accuracy ε ?
- Results of this kind are known as ***complexity bounds*** and typically require more information on the target function f (e.g. $f \in C^k(\mathbb{R})$ or $f \in W^{k,p}(\mathbb{R})$).

6.4 Complexity bounds for shallow networks

- **Question:** in practice, for a shallow neural network with architecture $(1, N, 1)$, how many hidden neurons N do I need to approximate f with a given accuracy ε ?
- Results of this kind are known as **complexity bounds** and typically require more information on the target function f (e.g. $f \in C^k(\mathbb{R})$ or $f \in W^{k,p}(\mathbb{R})$).

Theorem 2 (Complexity bound (shallow ReLU case))

Let $f \in C^2([0, 1])$ and define $\text{ReLU}(x) = \max\{0, x\}$, then for any $N \in \mathbb{N}$ there exists $f_N \in \mathcal{H}_{(1,N+1,1)}^{\text{ReLU}}$ such that

$$\|f - f_N\|_{\infty, [0,1]} \leq \frac{1}{8N^2} \|f^{(2)}\|_{\infty, [0,1]},$$

where $\|\cdot\|_{\infty, K}$ denotes the supremum norm on a compact K .

6.4 Complexity bounds for shallow networks

- **Question:** in practice, for a shallow neural network with architecture $(1, N, 1)$, how many hidden neurons N do I need to approximate f with a given accuracy ε ?
- Results of this kind are known as **complexity bounds** and typically require more information on the target function f (e.g. $f \in C^k(\mathbb{R})$ or $f \in W^{k,p}(\mathbb{R})$).

Theorem 2 (Complexity bound (shallow ReLU case))

Let $f \in C^2([0, 1])$ and define $\text{ReLU}(x) = \max\{0, x\}$, then for any $N \in \mathbb{N}$ there exists $f_N \in \mathcal{H}_{(1, N+1, 1)}^{\text{ReLU}}$ such that

$$\|f - f_N\|_{\infty, [0, 1]} \leq \frac{1}{8N^2} \|f^{(2)}\|_{\infty, [0, 1]},$$

where $\|\cdot\|_{\infty, K}$ denotes the supremum norm on a compact K .

Proof.

- On the grid $(x_j = j/N, -1 \leq j \leq N+1)$, define the following “hat” functions:

$$h_j(x) = \frac{\text{ReLU}(x - x_{j-1}) - \text{ReLU}(x - x_j)}{x_j - x_{j-1}} - \frac{\text{ReLU}(x - x_j) - \text{ReLU}(x - x_{j+1})}{x_{j+1} - x_j}$$

6.4 Complexity bounds for shallow networks

Proof.

- We make the following approximation: $f_N(x) = \sum_{j=0}^N f(x_j)h_j(x)$.

Then f_N is actually a linear interpolation of f .

6.4 Complexity bounds for shallow networks

Proof.

- We make the following approximation: $f_N(x) = \sum_{j=0}^N f(x_j)h_j(x)$.

Then f_N is actually a linear interpolation of f .

- On each interval $[x_j, x_{j+1}]$, we can bound the approximation error using the remainder formula for the Lagrange interpolating polynomial^a of degree one:

$$|f(x) - f_N(x)| = \frac{f^{(2)}(\xi)}{2} (x - x_j)(x_{j+1} - x), \quad \forall x \in [x_j, x_{j+1}], \text{ for some } \xi \in [x_j, x_{j+1}].$$

6.4 Complexity bounds for shallow networks

Proof.

- We make the following approximation: $f_N(x) = \sum_{j=0}^N f(x_j) h_j(x)$.

Then f_N is actually a linear interpolation of f .

- On each interval $[x_j, x_{j+1}]$, we can bound the approximation error using the remainder formula for the Lagrange interpolating polynomial^a of degree one:

$$|f(x) - f_N(x)| = \frac{f^{(2)}(\xi)}{2} (x - x_j)(x_{j+1} - x), \quad \forall x \in [x_j, x_{j+1}], \text{ for some } \xi \in [x_j, x_{j+1}].$$

Taking the supremum over $[x_j, x_{j+1}]$ and bounding $f^{(2)}$ by its supremum norm, we obtain:

$$\sup_{x \in [x_j, x_{j+1}]} |f(x) - f_N(x)| \leq \frac{\|f^{(2)}\|_{\infty, [0,1]}}{2} \left(\frac{x_{j+1} - x_j}{2} \right)^2 \leq \frac{1}{8N^2} \|f^{(2)}\|_{\infty, [0,1]}$$

^aSee the formulas in Chapter 25.2 of *Handbook of mathematical functions* by Abramowitz and Stegun.

6.4 Complexity bounds for shallow networks

Proof.

- If we re-arrange terms and define $f(x_{-1}) = f(x_{N+1}) = 0$, then

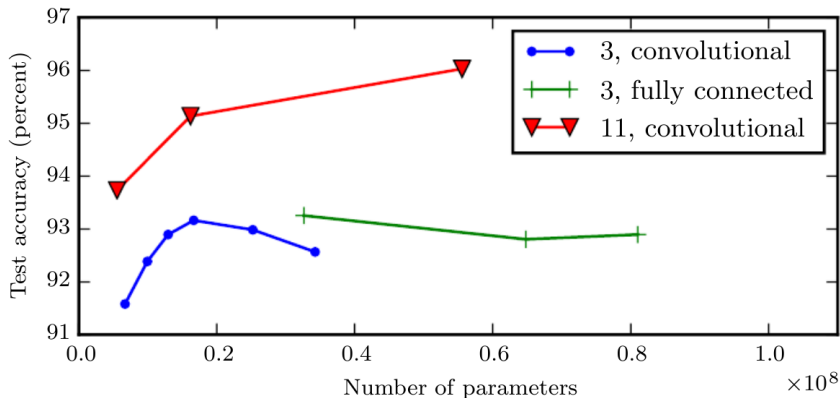
$$\begin{aligned} f_N(x) = & \sum_{j=0}^N N(f(x_{j+1}) - 2f(x_j) + f(x_{j-1}))\text{ReLU}(x - x_j) \\ & + Nf(0)\text{ReLU}(x - x_{-1}) + Nf(1)\text{ReLU}(x - x_{N+1}) \end{aligned}$$

and we see that f_N is actually a shallow ReLU network with $N + 1$ neurons (since the terms $\text{ReLU}(x - x_{N+1})$ and $\text{ReLU}(x - x_N)$ are identically zero).



6.5 Complexity bounds for shallow networks

- In practice, deep networks tend to perform better⁵ than shallow networks



⁵But remember that deeper networks are also more difficult to train!

Image taken from [Goodfellow et al., 2013]

6.5 Complexity bounds for deep networks

We mention (without proof) the following result:

Theorem 3 (Approximation of radial functions [Eldan and Shamir, 2016])

Let ρ be a ReLU or sigmoidal activation function. Then there exist constants $c, C > 0$ such that for every input dimension $d > C$ there is an input data distribution μ on $\mathbb{R}^d$ and a $G \in \mathcal{H}_{(d, N_1, N_2, 1)}^\rho$ with $\max\{N_1, N_2\} \leq Cd^5$, such that for every $n \leq ce^{cd}$ it holds that:

$$\inf_{F \in \mathcal{H}_{(d, n, 1)}^\rho} \|F - G\|_{L^2(\mu)} \geq c.$$

- Shallow networks require exponentially more neurons than deep networks to approximate certain functions (e.g. radial functions).

6.5 Complexity bounds for deep networks

- We now derive complexity bounds for deep neural networks.

6.5 Complexity bounds for deep networks

- We now derive complexity bounds for deep neural networks.
- The results holds for any activation ρ that satisfies the following assumptions:

A1) Approximation of $x \mapsto x$. For every $\varepsilon > 0$ there exists a neural network F_1^ε of width W_1^ε and depth L_1^ε such that $\sup_{x \in [-1,1]} |F_1^\varepsilon(x) - x| < \varepsilon$, and $\text{Lip}(F_1^\varepsilon) \leq 2$.

6.5 Complexity bounds for deep networks

- We now derive complexity bounds for deep neural networks.
 - The results holds for any activation ρ that satisfies the following assumptions:
- A1) Approximation of $x \mapsto x$. For every $\varepsilon > 0$ there exists a neural network F_1^ε of width W_1^ε and depth L_1^ε such that $\sup_{x \in [-1,1]} |F_1^\varepsilon(x) - x| < \varepsilon$, and $\text{Lip}(F_1^\varepsilon) \leq 2$.
- A2) Approximation of $x \mapsto x^2$. For every $\varepsilon > 0$ there exists a neural network F_2^ε of width W_2^ε and depth $L_2^\varepsilon = L_1^\varepsilon$ such that $\sup_{x \in [-1,1]} |F_2^\varepsilon(x) - x^2| < \varepsilon$, and $\text{Lip}(F_2^\varepsilon) \leq 2$.

6.5 Complexity bounds for deep networks

- We now derive complexity bounds for deep neural networks.
- The results holds for any activation ρ that satisfies the following assumptions:

- A1) Approximation of $x \mapsto x$. For every $\varepsilon > 0$ there exists a neural network F_1^ε of width W_1^ε and depth L_1^ε such that $\sup_{x \in [-1,1]} |F_1^\varepsilon(x) - x| < \varepsilon$, and $\text{Lip}(F_1^\varepsilon) \leq 2$.
- A2) Approximation of $x \mapsto x^2$. For every $\varepsilon > 0$ there exists a neural network F_2^ε of width W_2^ε and depth $L_2^\varepsilon = L_1^\varepsilon$ such that $\sup_{x \in [-1,1]} |F_2^\varepsilon(x) - x^2| < \varepsilon$, and $\text{Lip}(F_2^\varepsilon) \leq 2$.
- A3) Approximation of partition of unity. Let $N \in \mathbb{N}$, $0 \leq j \leq N$, $x_j^N = j/N$ and $I_j^N = [x_{j-1}^N, x_j^N]$. For every $\varepsilon > 0$ there exists a neural network $\Psi^\varepsilon : \mathbb{R} \rightarrow [0, 1]$ of width W_Ψ^ε and depth L_Ψ^ε such that for any $N \in \mathbb{N}$, $1 \leq j \leq N$ it holds for $\Psi_j^{N,\varepsilon} = \Psi^\varepsilon(N(x - x_j^N))$ that

$$\max_{x \in I_j^N} \left| \sum_{l=-1,0,1} \Psi_{j+l}^{N,\varepsilon}(x) - 1 \right| < \varepsilon \quad \text{and} \quad \max_{x \in I_j^N} \sum_{\substack{l \neq -1,0,1 \\ 1 \leq j+l \leq N}} \left| \Psi_{j+l}^{N,\varepsilon}(x) \right| < \varepsilon$$

6.5 Complexity bounds for deep networks

Theorem 4 (Complexity bounds for deep networks [De Ryck et al., 2021])

If assumptions (A1), (A2) and (A3) are satisfied, then for every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a neural network F^ε of width $O(\varepsilon^{-d/k}(W_1^\varepsilon + W_2^{\varepsilon(k+d)/k} + W_\Psi^\varepsilon))$ and depth $O(L_1^\varepsilon + L_2^{\varepsilon(k+d)/k} + L_\Psi^\varepsilon)$ such that

$$\sup_{x \in [0, 1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

6.5 Complexity bounds for deep networks

Theorem 4 (Complexity bounds for deep networks [De Ryck et al., 2021])

If assumptions (A1), (A2) and (A3) are satisfied, then for every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a neural network F^ε of width $O(\varepsilon^{-d/k}(W_1^\varepsilon + W_2^{\varepsilon(k+d)/k} + W_\Psi^\varepsilon))$ and depth $O(L_1^\varepsilon + L_2^{\varepsilon(k+d)/k} + L_\Psi^\varepsilon)$ such that

$$\sup_{x \in [0, 1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

Proof.

We prove the case $d = 1$ to keep notation simple, but the same argument applies for $d > 1$.

6.5 Complexity bounds for deep networks

Theorem 4 (Complexity bounds for deep networks [De Ryck et al., 2021])

If assumptions (A1), (A2) and (A3) are satisfied, then for every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a neural network F^ε of width $O(\varepsilon^{-d/k}(W_1^\varepsilon + W_2^{\varepsilon(k+d)/k} + W_\Psi^\varepsilon))$ and depth $O(L_1^\varepsilon + L_2^{\varepsilon(k+d)/k} + L_\Psi^\varepsilon)$ such that

$$\sup_{x \in [0,1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

Proof.

We prove the case $d = 1$ to keep notation simple, but the same argument applies for $d > 1$.

- **Main idea:** Let $N \in \mathbb{N}$, $0 \leq j \leq N$, $x_j^N = j/N$ and $I_j^N = [x_{j-1}^N, x_j^N]$.

By Taylor's theorem, for each I_j^N there exists a polynomial p_j^N with $\deg(p_j^N) \leq k - 1$ such that:

$$\|f - p_j^N\|_{\infty, I_j^N} \leq \frac{1}{k!} \left(\frac{1}{N}\right)^k \|f^{(k)}\|_{\infty, [0,1]}.$$

6.5 Complexity bounds for deep networks

Theorem 4 (Complexity bounds for deep networks [De Ryck et al., 2021])

If assumptions (A1), (A2) and (A3) are satisfied, then for every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a neural network F^ε of width $O(\varepsilon^{-d/k}(W_1^\varepsilon + W_2^{\varepsilon(k+d)/k} + W_\Psi^\varepsilon))$ and depth $O(L_1^\varepsilon + L_2^{\varepsilon(k+d)/k} + L_\Psi^\varepsilon)$ such that

$$\sup_{x \in [0,1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

Proof.

We prove the case $d = 1$ to keep notation simple, but the same argument applies for $d > 1$.

- **Main idea:** Let $N \in \mathbb{N}$, $0 \leq j \leq N$, $x_j^N = j/N$ and $I_j^N = [x_{j-1}^N, x_j^N]$.

By Taylor's theorem, for each I_j^N there exists a polynomial p_j^N with $\deg(p_j^N) \leq k - 1$ such that:

$$\|f - p_j^N\|_{\infty, I_j^N} \leq \frac{1}{k!} \left(\frac{1}{N}\right)^k \|f^{(k)}\|_{\infty, [0,1]}.$$

Let's approximate the p_j^N with suitable networks and let's use $\Psi_j^{N,\varepsilon}$ to approximate each I_j^N .

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**
 - First we approximate all monomials x^p , with $1 \leq p \leq k - 1$.

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**

- First we approximate all monomials x^p , with $1 \leq p \leq k - 1$.
- By (A1) and (A2), we know that $F_1^\varepsilon \approx x$ and $F_2^\varepsilon \approx x^2$.

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**

- First we approximate all monomials x^p , with $1 \leq p \leq k-1$.
- By (A1) and (A2), we know that $F_1^\varepsilon \approx x$ and $F_2^\varepsilon \approx x^2$.
- For higher-order monomials, we first approximate multiplication $m(x, y) = xy = \frac{1}{4}((x+y)^2 - (x-y)^2)$ with the neural network:

$$\hat{m}^\delta(x, y) := M^2 \left(F_2^\delta \left(\frac{x+y}{2M} \right) - F_2^\delta \left(\frac{x-y}{2M} \right) \right), \quad x, y \in [-M, M].$$

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**

- First we approximate all monomials x^p , with $1 \leq p \leq k-1$.
- By (A1) and (A2), we know that $F_1^\varepsilon \approx x$ and $F_2^\varepsilon \approx x^2$.
- For higher-order monomials, we first approximate multiplication $m(x, y) = xy = \frac{1}{4}((x+y)^2 - (x-y)^2)$ with the neural network:

$$\hat{m}^\delta(x, y) := M^2 \left(F_2^\delta \left(\frac{x+y}{2M} \right) - F_2^\delta \left(\frac{x-y}{2M} \right) \right), \quad x, y \in [-M, M].$$

- Then define recursively:

$$F_p^\varepsilon := \hat{m}^\varepsilon(F_{p-1}^\varepsilon(x), F_1^\varepsilon(x)) \approx x^p.$$

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**

- First we approximate all monomials x^p , with $1 \leq p \leq k-1$.
- By (A1) and (A2), we know that $F_1^\varepsilon \approx x$ and $F_2^\varepsilon \approx x^2$.
- For higher-order monomials, we first approximate multiplication $m(x, y) = xy = \frac{1}{4}((x+y)^2 - (x-y)^2)$ with the neural network:

$$\hat{m}^\delta(x, y) := M^2 \left(F_2^\delta \left(\frac{x+y}{2M} \right) - F_2^\delta \left(\frac{x-y}{2M} \right) \right), \quad x, y \in [-M, M].$$

- Then define recursively:

$$F_p^\varepsilon := \hat{m}^\varepsilon(F_{p-1}^\varepsilon(x), F_1^\varepsilon(x)) \approx x^p.$$

- We can make sure that all F_p^ε , $1 \leq p \leq k-1$ have the same depth of $\max\{1, (k-2)\}L_1^\varepsilon$ by "padding" with the identity network F_1^ε .

6.5 Complexity bounds for deep networks

Proof.

- **Construction of approximation:**

- First we approximate all monomials x^p , with $1 \leq p \leq k-1$.
- By (A1) and (A2), we know that $F_1^\varepsilon \approx x$ and $F_2^\varepsilon \approx x^2$.
- For higher-order monomials, we first approximate multiplication $m(x, y) = xy = \frac{1}{4}((x+y)^2 - (x-y)^2)$ with the neural network:

$$\hat{m}^\delta(x, y) := M^2 \left(F_2^\delta \left(\frac{x+y}{2M} \right) - F_2^\delta \left(\frac{x-y}{2M} \right) \right), \quad x, y \in [-M, M].$$

- Then define recursively:

$$F_p^\varepsilon := \hat{m}^\varepsilon(F_{p-1}^\varepsilon(x), F_1^\varepsilon(x)) \approx x^p.$$

- We can make sure that all F_p^ε , $1 \leq p \leq k-1$ have the same depth of $\max\{1, (k-2)\}L_1^\varepsilon$ by "padding" with the identity network F_1^ε .
- We approximate $p_j^N = \sum_{p=0}^{k-1} a_p x^p$ by the neural network $q_j^N(x) = \sum_{p=0}^{k-1} a_p F_p^\varepsilon(x)$. By (A1) and (A2), $\|x^p - F_p^\varepsilon\|_{\infty, [-1, 1]} \leq \max\{1, (k-2)\}2^{k-2}\varepsilon$, therefore $\|p_j^N - q_j^N\|_{\infty, [0, 1]} \leq C\varepsilon$. Our final approximation is:

$$\hat{F}^N := \sum_{j=1}^N \hat{m}^\delta(q_j^N(x), \Psi_j^N(x)) \approx f(x).$$

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

We bound the approximation error on each I_i^N as follows:

$$\begin{aligned} \|f - \hat{F}^N\|_{\infty, I_i^N} &\leq \underbrace{\left\| f - \sum_{j=1}^N f \cdot \Psi_j^N \right\|_{\infty, I_i^N}}_{(A)} + \underbrace{\left\| \sum_{j=1}^N (f - q_j^N) \cdot \Psi_j^N \right\|_{\infty, I_i^N}}_{(B)} \\ &\quad + \underbrace{\left\| \sum_{j=1}^N q_j^N \cdot \Psi_j^N - \sum_{j=1}^N \hat{m}^\delta(q_j^N, \Psi_j^N) \right\|_{\infty, I_i^N}}_{(C)} \end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (A)**

$$\begin{aligned}\left\|f - \sum_{j=1}^N f \cdot \Psi_j^N\right\|_{\infty, I_i^N} &\leq \left\|f - \sum_{l=-1,0,1} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} + \left\|\sum_{\substack{1 \leq l \leq n \\ l \neq -1,0,1}} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} \\ &\leq 2\|f\|_{\infty, [0,1]} \varepsilon \quad (\text{by (A3)})\end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (A)**

$$\begin{aligned}\left\|f - \sum_{j=1}^N f \cdot \Psi_j^N\right\|_{\infty, I_i^N} &\leq \left\|f - \sum_{l=-1,0,1} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} + \left\|\sum_{\substack{1 \leq l \leq n \\ l \neq -1,0,1}} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} \\ &\leq 2\|f\|_{\infty, [0,1]} \varepsilon \quad (\text{by (A3)})\end{aligned}$$

- **Bound on (C)**

$$\left\|\sum_{j=1}^N q_j^N \cdot \Psi_j^N - \sum_{j=1}^N \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N} \leq \sum_{j=1}^N \left\|q_j^N \cdot \Psi_j^N - \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (A)**

$$\begin{aligned}\left\|f - \sum_{j=1}^N f \cdot \Psi_j^N\right\|_{\infty, I_i^N} &\leq \left\|f - \sum_{l=-1,0,1} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} + \left\|\sum_{\substack{1 \leq l \leq n \\ l \neq -1,0,1}} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} \\ &\leq 2\|f\|_{\infty, [0,1]} \varepsilon \quad (\text{by (A3)})\end{aligned}$$

- **Bound on (C)**

$$\begin{aligned}\left\|\sum_{j=1}^N q_j^N \cdot \Psi_j^N - \sum_{j=1}^N \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N} &\leq \sum_{j=1}^N \left\|q_j^N \cdot \Psi_j^N - \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N} \\ &\leq N\|m - \hat{m}^\delta\|_{\infty, [-M, M]^2} \quad (\text{for big enough } M > 0)\end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (A)**

$$\begin{aligned}\left\|f - \sum_{j=1}^N f \cdot \Psi_j^N\right\|_{\infty, I_i^N} &\leq \left\|f - \sum_{l=-1,0,1} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} + \left\|\sum_{\substack{1 \leq l \leq n \\ l \neq -1,0,1}} f \cdot \Psi_{i+l}^N\right\|_{\infty, I_i^N} \\ &\leq 2\|f\|_{\infty, [0,1]} \varepsilon \quad (\text{by (A3)})\end{aligned}$$

- **Bound on (C)**

$$\begin{aligned}\left\|\sum_{j=1}^N q_j^N \cdot \Psi_j^N - \sum_{j=1}^N \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N} &\leq \sum_{j=1}^N \left\|q_j^N \cdot \Psi_j^N - \hat{m}^\delta(q_j^N, \Psi_j^N)\right\|_{\infty, I_i^N} \\ &\leq N \|m - \hat{m}^\delta\|_{\infty, [-M, M]^2} \quad (\text{for big enough } M > 0) \\ &\leq N \cdot 2M^2 \sup_{x \in [-1, 1]} |F_2^\delta(x) - x^2| \leq N \cdot 2M^2 \delta \\ &\quad (\text{by def. of } \hat{m}^\delta \text{ and (A2)})\end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (B)**

$$(B) \leq \left\| \sum_{l=-1,0,1}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N} + \left\| \sum_{\substack{1 \leq i+l \leq N \\ l \neq -1,0,1}}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (B)**

$$\begin{aligned}(B) &\leq \left\| \sum_{l=-1,0,1}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N} + \left\| \sum_{\substack{1 \leq i+l \leq N \\ l \neq -1,0,1}}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N} \\ &\leq \max_{l=-1,0,1} \|f - q_{i+l}^N\|_{\infty, I_i^N} \cdot \left\| \sum_{l=-1,0,1}^N \Psi_{i+l}^N \right\|_{\infty, I_i^N} + \max_{1 \leq j \leq N} \|f - q_j^N\|_{\infty, I_i^N} \cdot \left\| \sum_{\substack{1 \leq i+l \leq N \\ l \neq -1,0,1}}^N \Psi_{i+l}^N \right\|_{\infty, I_i^N}\end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bound on (B)**

$$\begin{aligned}(B) &\leq \left\| \sum_{l=-1,0,1}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N} + \left\| \sum_{\substack{1 \leq i+l \leq N \\ l \neq -1,0,1}}^N (f - q_{i+l}^N) \cdot \Psi_{i+l}^N \right\|_{\infty, I_i^N} \\ &\leq \max_{l=-1,0,1} \|f - q_{i+l}^N\|_{\infty, I_i^N} \cdot \left\| \sum_{l=-1,0,1}^N \Psi_{i+l}^N \right\|_{\infty, I_i^N} + \max_{1 \leq j \leq N} \|f - q_j^N\|_{\infty, I_i^N} \cdot \left\| \sum_{\substack{1 \leq i+l \leq N \\ l \neq -1,0,1}}^N \Psi_{i+l}^N \right\|_{\infty, I_i^N} \\ &\leq \left(\frac{\|f^{(k)}\|_{\infty, [0,1]}}{k!} \left(\frac{3}{N} \right)^k + C\varepsilon \right) (1 + \varepsilon) + \left(\frac{\|f^{(k)}\|_{\infty, [0,1]}}{k!} + C\varepsilon \right) \cdot \varepsilon\end{aligned}$$

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

To obtain $O(\varepsilon)$ error we set $N = \varepsilon^{-1/k}$ and $\delta = \varepsilon^{(k+1)/k}$.

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

To obtain $O(\varepsilon)$ error we set $N = \varepsilon^{-1/k}$ and $\delta = \varepsilon^{(k+1)/k}$.

- **Total width and depth**

- Polynomials q_j^N require a network of width $O(N(W_1^\varepsilon + W_2^\varepsilon))$ and depth $O(L_1^\varepsilon)$.

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

To obtain $O(\varepsilon)$ error we set $N = \varepsilon^{-1/k}$ and $\delta = \varepsilon^{(k+1)/k}$.

- **Total width and depth**

- Polynomials q_j^N require a network of width $O(N(W_1^\varepsilon + W_2^\varepsilon))$ and depth $O(L_1^\varepsilon)$.
- The Ψ_j^N require a parallel network of width $O(NW_\Psi^\varepsilon)$ and depth $O(L_\Psi^\varepsilon)$.

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

To obtain $O(\varepsilon)$ error we set $N = \varepsilon^{-1/k}$ and $\delta = \varepsilon^{(k+1)/k}$.

- **Total width and depth**

- Polynomials q_j^N require a network of width $O(N(W_1^\varepsilon + W_2^\varepsilon))$ and depth $O(L_1^\varepsilon)$.
- The Ψ_j^N require a parallel network of width $O(NW_\Psi^\varepsilon)$ and depth $O(L_\Psi^\varepsilon)$.
- For the multiplication of q_j^N and Ψ_j^N we need a network of width $O(NW_2^\delta)$ and depth $O(L_2^\delta)$.

6.5 Complexity bounds for deep networks

Proof.

- **Bounding the error**

Putting all bounds together:

$$\|f - \hat{F}^N\|_{\infty, [0,1]} \leq \max_{1 \leq i \leq N} \|f - \hat{F}^N\|_{\infty, I_i^N} \leq C(N^{-k} + \varepsilon + N\delta)$$

To obtain $O(\varepsilon)$ error we set $N = \varepsilon^{-1/k}$ and $\delta = \varepsilon^{(k+1)/k}$.

- **Total width and depth**

- Polynomials q_j^N require a network of width $O(N(W_1^\varepsilon + W_2^\varepsilon))$ and depth $O(L_1^\varepsilon)$.
- The Ψ_j^N require a parallel network of width $O(NW_\Psi^\varepsilon)$ and depth $O(L_\Psi^\varepsilon)$.
- For the multiplication of q_j^N and Ψ_j^N we need a network of width $O(NW_2^\delta)$ and depth $O(L_2^\delta)$.

The total width is $O(N(W_1^\varepsilon + W_2^\delta + W_\Psi^\varepsilon))$ and total depth is $O(L_1^\varepsilon + L_2^\delta + L_\Psi^\varepsilon)$.



6.5 Complexity bounds for deep networks

In the case of ReLU networks (see Problem Set 4, Ex 2 and 3):

A1) $W_1^\varepsilon = 2, L_1^\varepsilon = 2,$

A2) $W_2^\varepsilon = O(\log(1/\varepsilon)), L_2^\varepsilon = O(\log(1/\varepsilon)),$

A3) $W_\Psi^\varepsilon = 3, L_\Psi^\varepsilon = 2.$

6.5 Complexity bounds for deep networks

In the case of ReLU networks (see Problem Set 4, Ex 2 and 3):

A1) $W_1^\varepsilon = 2, L_1^\varepsilon = 2,$

A2) $W_2^\varepsilon = O(\log(1/\varepsilon)), L_2^\varepsilon = O(\log(1/\varepsilon)),$

A3) $W_\Psi^\varepsilon = 3, L_\Psi^\varepsilon = 2.$

Theorem 5 (Complexity bounds for deep ReLU networks)

For every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a ReLU neural network F^ε with width $O(\varepsilon^{-d/k} \log(1/\varepsilon))$ and depth $O(\log(1/\varepsilon))$ such that

$$\sup_{x \in [0, 1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

6.5 Complexity bounds for deep networks

In the case of ReLU networks (see Problem Set 4, Ex 2 and 3):

A1) $W_1^\varepsilon = 2, L_1^\varepsilon = 2,$

A2) $W_2^\varepsilon = O(\log(1/\varepsilon)), L_2^\varepsilon = O(\log(1/\varepsilon)),$

A3) $W_\Psi^\varepsilon = 3, L_\Psi^\varepsilon = 2.$

Theorem 5 (Complexity bounds for deep ReLU networks)

For every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a ReLU neural network F^ε with width $O(\varepsilon^{-d/k} \log(1/\varepsilon))$ and depth $O(\log(1/\varepsilon))$ such that

$$\sup_{x \in [0, 1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

Remarks:

- A shallow ReLU network requires $O(\varepsilon^{-1/2})$ to approximate $x \mapsto x^2$ (see Theorem 2), while a deep ReLU network requires only $O(\log(1/\varepsilon))$ (by (A2)),

6.5 Complexity bounds for deep networks

In the case of ReLU networks (see Problem Set 4, Ex 2 and 3):

A1) $W_1^\varepsilon = 2, L_1^\varepsilon = 2,$

A2) $W_2^\varepsilon = O(\log(1/\varepsilon)), L_2^\varepsilon = O(\log(1/\varepsilon)),$

A3) $W_\Psi^\varepsilon = 3, L_\Psi^\varepsilon = 2.$

Theorem 5 (Complexity bounds for deep ReLU networks)

For every $f \in C^k([0, 1]^d)$ and $\varepsilon > 0$ there exists a ReLU neural network F^ε with width $O(\varepsilon^{-d/k} \log(1/\varepsilon))$ and depth $O(\log(1/\varepsilon))$ such that

$$\sup_{x \in [0, 1]^d} |f(x) - F^\varepsilon(x)| < \varepsilon.$$

Remarks:

- A shallow ReLU network requires $O(\varepsilon^{-1/2})$ to approximate $x \mapsto x^2$ (see Theorem 2), while a deep ReLU network requires only $O(\log(1/\varepsilon))$ (by (A2)),
- The exponential dependence in d is known as the **curse of dimensionality** (CoD).

Conclusions

Take-home messages:

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,
- we can use complexity bounds to estimate how the number of parameters scales with the approximation accuracy.

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,
- we can use complexity bounds to estimate how the number of parameters scales with the approximation accuracy.

But...

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,
- we can use complexity bounds to estimate how the number of parameters scales with the approximation accuracy.

But...

- UAP proofs ultimately rely on the Stone-Weierstrass theorem,
⇒ they don't really tell us what's so “special” about neural networks.

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,
- we can use complexity bounds to estimate how the number of parameters scales with the approximation accuracy.

But...

- UAP proofs ultimately rely on the Stone-Weierstrass theorem,
 $\implies$ they don't really tell us what's so "special" about neural networks.
- UAP results and complexity bounds show that a good neural network approximation exists, but they cannot be used to construct it in practice.

Conclusions

Take-home messages:

- neural networks are universal approximators for functions,
- we can use complexity bounds to estimate how the number of parameters scales with the approximation accuracy.

But...

- UAP proofs ultimately rely on the Stone-Weierstrass theorem,
 $\implies$ they don't really tell us what's so "special" about neural networks.
- UAP results and complexity bounds show that a good neural network approximation exists, but they cannot be used to construct it in practice.
- So far we have ***no guarantee*** that a sufficiently wide and deep network will approximate well ***after training***, due to many sources of error: finite sample size, SGD optimization is local and noisy, the loss function is not convex ...

References I

-  Bengio, Y., Lamblin, P., Popovici, D., and Larochelle, H. (2006). Greedy layer-wise training of deep networks. *Advances in neural information processing systems*, 19.
-  Berner, J., Grohs, P., Kutyniok, G., and Petersen, P. (2021). The modern mathematics of deep learning. *arXiv preprint arXiv:2105.04026*, pages 86–114.
-  Bernstein, L., Sludds, A., Hamerly, R., Sze, V., Emer, J., and Englund, D. (2021). Freely scalable and reconfigurable optical hardware for deep learning. *Scientific reports*, 11(1):3144.
-  Cheridito, P. and Gersey, B. (2023). Computation of conditional expectations with guarantees. *Journal of Scientific Computing*, 95(1):12.
-  Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314.

References II

-  De Ryck, T., Lanthaler, S., and Mishra, S. (2021).
On the approximation of functions by tanh neural networks.
Neural Networks, 143:732–750.
-  Eldan, R. and Shamir, O. (2016).
The power of depth for feedforward neural networks.
In *Conference on learning theory*, pages 907–940. PMLR.
-  Goodfellow, I. J., Bulatov, Y., Ibarz, J., Arnoud, S., and Shet, V. (2013).
Multi-digit number recognition from street view imagery using deep convolutional neural networks.
arXiv preprint arXiv:1312.6082.
-  Hebb, D. O. (2005).
The organization of behavior: A neuropsychological theory.
Psychology press.

References III

-  Hinton, G. E., Osindero, S., and Teh, Y.-W. (2006).
A fast learning algorithm for deep belief nets.
Neural computation, 18(7):1527–1554.
-  Hochreiter, S. and Schmidhuber, J. (1997).
Long short-term memory.
Neural computation, 9(8):1735–1780.
-  Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993).
Multilayer feedforward networks with a nonpolynomial activation function can approximate any function.
Neural networks, 6(6):861–867.
-  McCulloch, W. S. and Pitts, W. (1943).
A logical calculus of the ideas immanent in nervous activity.
The bulletin of mathematical biophysics, 5:115–133.

References IV

-  Ranzato, M., Poultney, C., Chopra, S., and Cun, Y. (2006).
Efficient learning of sparse representations with an energy-based model.
Advances in neural information processing systems, 19.
-  Rosenblatt, F. (1958).
The perceptron: a probabilistic model for information storage and organization in the brain.
Psychological review, 65(6):386.
-  Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986).
Learning representations by back-propagating errors.
nature, 323(6088):533–536.